Word Embedding Based E-Commerce Semantic Search Engine

Abstract

This report presents summary of an e-commerce semantic search engine built on the Flipkart product corpus. The system converts heterogeneous catalog text into dense sentence embeddings using the `all-MiniLM-L12-v2` SentenceTransformer, indexes products with FAISS, and exposes retrieval through a Flask API. Instead of relying on a single retrieval strategy, the project benchmarks four complementary models: a hierarchical graph-based model (Model 1), a flat semantic baseline (Model 2), an enhanced semantic model with metadata-aware re-ranking (Model 3), and an LLM-assisted query expansion model (Model 4). The cleaned corpus contains 12,732 products derived from 20,000 raw records and standardized into an 18-column schema. Benchmarks across exact, semantic, and vague conversational queries show clear trade-offs: Model 2 is the fastest and strongest on exact queries, Model 3 achieves the best semantic-search quality (NDCG 0.897 on semantic queries), Model 4 improves vague-query recall at significant latency cost, and the ensemble produces the best overall semantic NDCG (0.904) but is too slow for default deployment. Based on these results, the project adopts the enhanced semantic pipeline as the practical default because it provides the strongest quality-latency balance.

1 Introduction and Motivation

Product search in real e-commerce systems is fundamentally harder than exact keyword lookup. Users rarely search with clean catalog labels; instead they use natural language such as *"something to watch movies on"* or *"comfortable athletic footwear for marathon training"*. A keyword-only system is brittle in this setting because lexical mismatch between query and catalog vocabulary directly reduces recall.

The objective of this project is to move from rigid lexical retrieval to intent-aware semantic retrieval. The system should understand that *"watch movies"* may imply televisions, projectors, monitors, or home-theatre products; that *"carry laptop"* implies laptop bags or backpacks; and that explicitly constrained queries such as *"men's tshirt"* should not surface unrelated apparel.

To address this, the project implements a multi-model semantic search architecture with three goals:

(1) **Representation:** convert noisy product text into vector embeddings that preserve semantic similarity.
(2) **Retrieval:** support both structured and conversational search through multiple search strategies.
(3) **Evaluation:** benchmark quality and latency across exact, semantic, and vague query classes.

2 Background

2.1 Semantic Search

Semantic search replaces exact string matching with embedding-based retrieval. A text encoder maps both product descriptions and queries into a shared vector space, where semantically similar items lie closer together. Sentence-BERT style encoders are effective for this setup because they generate sentence-level dense representations suitable for similarity search.

2.2 Vector Indexing with FAISS

Given a query embedding $q \in \mathbb{R}^d$ and product embeddings $\{x_i\}_{i=1}^N$, retrieval reduces to nearest-neighbor search under a similarity or distance function. The project uses FAISS for scalable vector retrieval. Since the processed corpus is modest in size, the code switches to an exact flat index below a threshold of 50,000 items.

2.3 Similarity and Re-Ranking

Two core scoring ideas appear throughout the project. For the hierarchical model, cosine similarity is computed between query embeddings and node embeddings:

$$\cos(q, x) = \frac{q \cdot x}{\|q\|\|x\|}. \tag{1}$$

For FAISS-backed models, L2 distance is converted to a confidence score

$$s_{\text{semantic}} = \max\left(0, \min\left(1, 1 - \frac{d_{\text{L2}}}{2}\right)\right). \tag{2}$$

The enhanced model then combines semantic relevance with metadata signals using a weighted linear score.

3 Dataset and Preprocessing

3.1 Dataset

The project uses the Flipkart e-commerce sample dataset, containing 20,000 raw product records. The raw dataset includes semi-structured fields such as product name, category tree, free-text

Table 1: Dataset summary after the project preprocessing pipeline.

Property	Value
Raw records	20,000
Processed records	12,732
Processed schema columns	18
Enriched schema columns	19
Embedding model dimension	384
Default top-k	5

description, product specifications, pricing information, and partial brand metadata.

The raw catalog is transformed into a unified schema with 18 fields, including `product_id`, `product_name`, `department`, `category`, `subcategory`, `product_type`, `description`, `features`, `price`, `rating`, `reviews_count`, `tags`, `color`, `size_range`, `material`, `brand`, `stock_status`, and `keywords`. After preprocessing and deduplication, the working corpus contains 12,732 products.

3.2 Loading and Schema Transformation

The loader parses Flipkart's category tree into hierarchical fields and extracts structured product information from the raw `product_specifications` field. Missing or inconsistent fields are normalized into a consistent tabular representation. Price is recovered from the best available pricing column, tags are synthesized from category tokens, and fallback values are inserted for missing brand or stock metadata.

3.3 Preprocessing Pipeline

The preprocessing pipeline performs the following operations:

(1) text cleaning by removing HTML, URLs, excess whitespace, and non-ASCII artifacts;
(2) boilerplate stripping from long description fields;
(3) price normalization with extreme outlier capping;
(4) rating normalization with clipping to the range $[0, 5]$;
(5) duplicate removal using both `product_id` and near-duplicate `product_name + category` checks;
(6) invalid-row removal for empty or malformed products;
(7) missing-value imputation for categorical and descriptive fields.

This is important because semantic retrieval quality depends directly on embedding quality. Dirty descriptions, duplicated records, and inconsistent category fields would otherwise distort the vector space.

3.4 Description Enrichment

The enriched pipeline adds an `enriched_description` column using department-aware templates. Rather than embedding only sparse product titles or short descriptions, the system synthesizes richer product narratives that combine:

- product name and type,
- category and subcategory context,
- brand, price, rating, and tags,
- extracted material, color, and size metadata,
- use-case language based on department or category.

Although the current implementation guarantees roughly 50+ words rather than the longer descriptions proposed in presentation material, the enriched field still provides denser semantic context than the raw catalog text and is the basis of Model 3 and Model 4 retrieval.

4 System Architecture

4.1 Pipeline Overview

The project follows a layered architecture:

(1) **Data layer:** raw CSV ingestion, schema transformation, cleaning, and enrichment.
(2) **Embedding layer:** SentenceTransformer-based encoding with `all-MiniLM-L12-v2`.
(3) **Index layer:** FAISS for dense retrieval and NetworkX for hierarchical graph traversal.
(4) **Model layer:** four retrieval models with distinct scoring logic.
(5) **Orchestration layer:** query routing, intent-aware score adjustment, and weighted ensemble aggregation.
(6) **Serving layer:** Flask REST API endpoints for search, browse, evaluation, health checks, and comparison UI.

The application starts by loading processed data if available; otherwise it regenerates cleaned and enriched datasets, builds model indices, and then exposes the API through Flask.

4.2 API Surface

The implementation provides:

- `POST /api/search` for model-specific or default semantic search,
- `POST /api/search/{model_id}` for direct model routing,
- `GET /api/products` and `GET /api/products/{product_id}` for browsing,
- `POST /api/evaluate` for running benchmark suites,
- `GET /api/health` and `GET /api/ready` for system monitoring.

5 Models and Methods

5.1 Model 1: Hierarchical Graph Search

Model 1 constructs a directed hierarchy from department to product type and product leaf nodes using NetworkX. Non-leaf nodes are embedded from hierarchy labels, while product nodes are embedded from product name, category, and feature text.

Query processing follows a top-down traversal. At each level, the query is compared against candidate nodes using cosine similarity and only the strongest branches are expanded. The final product score is a decayed aggregation across hierarchy levels:

$$S_{\text{hier}} = \frac{s_1\gamma^4 + s_2\gamma^3 + s_3\gamma^2 + s_4\gamma + s_5}{5}, \tag{3}$$

where $s_1, \ldots, s_5$ are similarities from department through product level and $\gamma = 0.85$ is the hierarchy decay factor.

This model is interpretable and structurally aligned with catalog taxonomies, but it is slower and can be too rigid for cross-category intent.

5.2 Model 2: Flat Semantic Search

Model 2 ignores hierarchy and encodes each product as a single combined text description assembled from key product attributes.

Retrieval is performed directly with FAISS over the embedding matrix. Because no metadata or taxonomy filtering is applied, this model offers strong speed and broad semantic coverage but can surface noisy results.

5.3 Model 3: Enhanced Semantic Search

Model 3 extends Model 2 in two ways:

(1) it encodes enriched descriptions instead of sparse combined text;
(2) it re-ranks an over-fetched candidate pool using metadata signals.

For a retrieved candidate, the final score is

$$S_{\text{enh}} = 0.85\, s_{\text{semantic}} + 0.05\, s_{\text{rating}} + 0.02\, s_{\text{reviews}} + 0.03\, s_{\text{price}} + 0.05\, s_{\text{stock}}. \tag{4}$$

The re-ranking design reflects the actual repository configuration: semantic similarity dominates because the Flipkart corpus contains sparse or weakly informative review and stock fields. This model is the project default because it produces strong retrieval quality while preserving millisecond-level latency.

5.4 Model 4: LLM-Enhanced Query Refinement

Model 4 inherits Model 3's index but prepends a query-refinement stage. Vague or conversational inputs are passed to an LLM via the OpenAI-compatible client configured against OpenRouter. The system prompt asks the model to convert ambiguous user intent into more precise product-oriented search phrases.

If the LLM call fails due to missing credentials, timeout, or provider issues, the model falls back to a deterministic rule-based expansion table. For example:

- *watch movies* → *television tv monitor projector home theatre*
- *keep money* → *wallet purse cardholder*
- *carry laptop* → *laptop bag backpack messenger*

This significantly helps vague-query recall, but the external API call introduces substantial latency.

5.5 Search Orchestrator and Intent Constraints

The orchestrator applies lightweight lexical constraints before or after model retrieval. It detects explicit intents such as gender (*men*, *women*), t-shirt-specific wording, and child-oriented phrases. Results that violate explicit intent are deterministically down-weighted. This is particularly useful for fashion queries, where semantic similarity alone can retrieve visually or contextually adjacent but incorrect items.

5.6 Ensemble Retrieval

The ensemble runs all available models and aggregates results by weighted voting:

$$S_{\text{ensemble}} = 0.20\, S_1 + 0.20\, S_2 + 0.35\, S_3 + 0.25\, S_4. \tag{5}$$

Duplicate products are merged by `product_id`, scores are summed across model votes, and top-k products are returned after re-ranking.

6 Evaluation Setup

6.1 Benchmark Protocol

The repository evaluates search quality using three curated query sets:

- **Exact queries** where users mention clear product types or brands;
- **Semantic queries** where intent is descriptive but not lexically identical to the catalog;
- **Vague queries** where users express a need without naming the product.

The benchmark suite contains 15 exact queries, 13 semantic queries, and 14 vague queries. Relevance is defined through keyword matching against returned product metadata fields such as department, category, subcategory, product type, and product name.

6.2 Metrics

The code computes Precision@5, Recall@5, Mean Reciprocal Rank (MRR), NDCG@5, F1 score, and average query latency. Since the benchmark uses category-keyword relevance instead of exhaustive product-level labels, the implementation treats recall in a bounded top-k manner based on retrieved hits rather than the total number of relevant products in the corpus. This makes the benchmark practical for the available dataset, but the recall numbers should be interpreted as relative model indicators rather than absolute corpus recall.

7 Results

7.1 Per-Query-Type Performance

Table 2 summarizes the benchmark output captured in `results/evaluation_report.json`.

7.2 Aggregate Comparison

To simplify comparison, Table 3 reports macro-averaged metrics across the three query groups.

7.3 Observations

The benchmark reveals four concrete patterns:

(1) **Model 2 dominates exact queries.** Its sparse combined text representation appears sufficient when user wording is already close to the catalog vocabulary, and it is by far the fastest model.
(2) **Model 3 is the best semantic default.** It achieves the strongest semantic-query F1 and matches the top semantic-query NDCG while staying close to Model 2 in latency.
(3) **Model 4 helps vague intent, but latency is prohibitive.** It delivers the best vague-query recall and NDCG, showing that query expansion improves ambiguous retrieval, but average latency rises to more than 1.3 seconds.
(4) **The ensemble improves ranking quality but is not deployment-friendly.** It produces the highest semantic NDCG and MRR, yet it is slower than any individual model because it executes all pipelines.

Table 2: Benchmark results for all models across query types. Times are average end-to-end latencies in milliseconds.

Model	Query Type	P@5	R@5	MRR	NDCG@5	F1	Time (ms)
Model 1 Hierarchical	Exact	0.480	0.600	0.567	0.579	0.523	532.09
Model 1 Hierarchical	Semantic	0.723	0.846	0.795	0.813	0.769	218.21
Model 1 Hierarchical	Vague	0.643	0.857	0.774	0.803	0.707	249.65
Model 2 Flat Semantic	Exact	**0.787**	**0.867**	**0.867**	**0.853**	**0.819**	**15.24**
Model 2 Flat Semantic	Semantic	0.800	0.923	0.846	0.883	0.845	16.70
Model 2 Flat Semantic	Vague	0.614	0.857	0.714	0.765	0.691	16.89
Model 3 Enhanced	Exact	0.653	0.800	0.733	0.755	0.702	19.06
Model 3 Enhanced	Semantic	**0.815**	**1.000**	0.859	0.897	**0.871**	17.46
Model 3 Enhanced	Vague	0.514	0.786	0.750	0.758	0.583	19.05
Model 4 LLM	Exact	0.653	0.800	0.733	0.755	0.702	1442.96
Model 4 LLM	Semantic	**0.815**	**1.000**	0.859	0.897	**0.871**	1340.34
Model 4 LLM	Vague	**0.629**	**0.929**	0.750	**0.801**	**0.715**	1384.81
Ensemble	Exact	0.747	0.867	0.800	0.816	0.794	1560.49
Ensemble	Semantic	0.785	**1.000**	**0.877**	**0.904**	0.842	1651.15
Ensemble	Vague	0.586	0.857	**0.764**	0.778	0.664	1625.24

Table 3: Macro-average performance across exact, semantic, and vague query sets.

Model	Avg. P@5	Avg. NDCG	Avg. Time (ms)
Model 1 Hierarchical	0.615	0.731	333.32
Model 2 Flat Semantic	**0.734**	**0.834**	**16.28**
Model 3 Enhanced	0.661	0.803	18.52
Model 4 LLM	0.699	0.818	1389.37
Ensemble	0.706	0.833	1612.29

8 Discussion

The most important engineering outcome is that retrieval quality is not determined solely by the sophistication of the model. The flat semantic baseline is hard to beat on exact-match style queries because the user already provides catalog-aligned terms. The enhanced semantic model then becomes valuable when richer context matters, since enrichment and re-ranking help the system recover better ordering on descriptive queries without sacrificing responsiveness.

The LLM-enhanced model demonstrates that vague-query expansion is useful, but it also highlights a practical systems constraint: an external LLM call can dominate latency. In the current implementation, Model 4 is best viewed as an optional refinement path for ambiguous user inputs rather than the default search backend.

The hierarchical model offers interpretability and category coherence, but the benchmark suggests that structure alone is not sufficient for heterogeneous product search. It is slower and less flexible than FAISS-backed approaches, particularly when the query spans multiple catalog branches.

8.1 Limitations

The current system has several limitations:

- relevance labels are keyword-derived rather than manually annotated;
- the enrichment pipeline is template-based rather than generative;
- some metadata signals, especially reviews and stock information, are sparse in the source corpus;
- the parsed category tree is noisy, which weakens the semantic value of top-level department statistics and hierarchical routing.

Future work could introduce manually labeled evaluation sets, better category normalization, hybrid lexical-semantic retrieval, and selective LLM invocation based on ambiguity detection instead of unconditional query refinement.

9 Conclusion

This project demonstrates a complete semantic-search stack for e-commerce, from raw catalog ingestion and preprocessing to embedding-based retrieval, multi-model benchmarking, and API deployment. The codebase shows that a practical production design does not require choosing between purely symbolic and purely neural approaches; instead, it benefits from combining structured metadata, dense retrieval, deterministic business rules, and optional LLM assistance.

From the measured results, the recommended default is Model 3, the enhanced semantic search pipeline. It preserves near-real-time latency, improves ranking quality on descriptive queries, and avoids the operational cost of external LLM dependency. Model 2 remains a strong fast baseline, Model 4 is useful for ambiguous conversational inputs, and the ensemble is best suited for offline analysis rather than serving traffic directly.

Acknowledgments

We thank the CS 328 teaching team for guidance throughout the project. We also acknowledge the open-source tools that made the system possible, including SentenceTransformers, FAISS, Flask, pandas, and NetworkX.

A Implementation Notes

A.1 Core Configuration

The repository uses the following notable configuration values:

- embedding model: `all-MiniLM-L12-v2`,
- embedding dimension: 384,
- FAISS index mode: flat search below 50,000 vectors,
- hierarchical decay factor: 0.85,
- reranking weights: semantic 0.85, rating 0.05, reviews 0.02, price 0.03, stock 0.05,
- default API port: 5000.

A.2 Query Set Sizes

Table 4: Benchmark query-set composition.

Query set	Count
Exact	15
Semantic	13
Vague	14
Total	42

References

[1] Miguel Grinberg. 2018. *Flask Web Development: Developing Web Applications with Python.* "O'Reilly Media, Inc.".

[2] Aric Hagberg, Pieter Swart, and Daniel S Schult. 2008. Exploring network structure, dynamics, and function using NetworkX. In *Proceedings of the 7th Python in Science Conference*. Pasadena, CA, 11–15.

[3] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[4] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics. https://arxiv.org/abs/1908.10084

[5] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. *Advances in Neural Information Processing Systems* 33 (2020), 5776–5788.